

Category B Practice Problem

PROBLEM 1 • Basic

Movie Ticket Field Visibility Checker

Problem Statement

CineHub's booking engine has a `MovieTicket` class with fields at every access level, and the team wants a small linter that checks a proposed access attempt against Java's four access levels before anyone wastes time trying to compile it.

Design `AccessChecker` to classify a single attempt, and build `MovieTicket` itself with the right access level on each field.

Requirements

- `classifyAccess(...)` must decide `ALLOWED` or `DENIED` purely from the real Java rules for `SAME_CLASS`, `SAME_PACKAGE`, and `DIFFERENT_PACKAGE`.
- Choose the right access level for each of `MovieTicket`'s four fields (`seatNumber`, `screenId`, `ticketPrice`, `movieTitle`) based on who genuinely needs to reach it.

Function Signature(s)

```
static String classifyAccess(String fieldModifier, String accessorContext)
static String summarizeBatch(String[][] attempts)
```

Examples

<code>classifyAccess("private", "SAME_CLASS")</code>	<code>"ALLOWED"</code>

<code>classifyAccess("protected", "DIFFERENT_PACKAGE")</code>	<code>"DENIED"</code>

<code>summarizeBatch({ {"default", "SAME_PACKAGE"}, {"default", "DIFFERENT_PACKAGE"}, {"public", "DIFFERENT_PACKAGE"} })</code>	<code>"Allowed: 2 Denied: 1"</code>

HINT Write out the four modifiers against the three contexts as a small table on paper first — the code almost writes itself once the table is right in front of you.

CONSTRAINTS `fieldModifier` ∈ {"private", "default", "protected", "public"}. `accessorContext` ∈ {"SAME_CLASS", "SAME_PACKAGE", "DIFFERENT_PACKAGE"}.

PROBLEM 2 • Basic → Intermediate

Subclass Ticket Access

Problem Statement

CineHub's loyalty module adds PremiumMovieTicket, extending MovieTicket from a different package to read its protected ticketPrice. Extend AccessChecker to handle this correctly — including the one case that trips almost everyone up.

Requirements

- Add accessorContext values SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE and SUBCLASS_DIFFERENT_PACKAGE_PARENT_TYPE.
- protected must be ALLOWED for the OWN_TYPE case, but DENIED for the PARENT_TYPE case — private and default stay DENIED for both.

Function Signature(s)

```
static String classifyAccess(String fieldModifier, String accessorContext) // now handles 5 contexts total
```

Examples

<code>classifyAccess("protected", "SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE")</code>	"ALLOWED"

<code>classifyAccess("protected", "SUBCLASS_DIFFERENT_PACKAGE_PARENT_TYPE")</code>	"DENIED"

Explanation: Even though the object really is a PremiumMovieTicket, accessing it through a plain MovieTicket-typed variable denies

protected access — Java goes by the reference's declared type.

HINT Ask "what type is the variable declared as?", not "what does the object actually turn out to be?" — that question alone decides the PARENT_TYPE case.

CONSTRAINTS Same modifier set as Problem 1, now with 5 possible accessorContext values.

Key Concepts Tested: *protected's cross-package rule with inheritance, compile-time vs. runtime typing.* Category B Problems Page 2

of 5

STEP SEM-3 · Week 6 · CodInClub | Powered by BridgeLabz

PROBLEM 3 • Intermediate

Seat Booking Encapsulation Guard

Problem Statement

A previous version of CineScreen let its available-seat count be set directly, and a bug once pushed it below zero during a weekend rush. Redesign CineScreen so seat counts only ever move through validated methods, and so it can never even be created with a nonsensical seat count.

Requirements

- seatsTotal and seatsAvailable must both be private, with no way to set seatsAvailable directly from outside.
- The constructor must reject a seatsTotal that's zero or negative.
- bookSeat() and cancelBooking() must each silently reject a transition that would push the count out of bounds.

Function Signature(s)

```
CineScreen(int seatsTotal)
void bookSeat()
void cancelBooking()
int getSeatsAvailable()
```

Examples

<code>new CineScreen(0)</code>	construction rejected

<pre>CineScreen c = new CineScreen(2); c.bookSeat(); c.bookSeat(); c.bookSeat(); c.getSeatsAvailable()</pre>	0

Explanation: The 3rd booking has no seat left to take — rejected, the count never goes negative.

<pre>c.cancelBooking(); c.cancelBooking(); c.cancelBooking(); c.getSeatsAvailable()</pre>	2

Explanation: The 3rd cancellation has nothing genuine to undo — the screen is already back to full.

HINT Think about the one relationship that must always hold true between `seatsAvailable` and `seatsTotal` — check it before every change, not just at the extremes.

CONSTRAINTS `seatsTotal`, once valid, is a positive integer up to 300.

Key Concepts Tested: Constructor-time validation, private fields with validated methods, boundary-safe state transitions.

PROBLEM 4 • Intermediate

MovieBookingProfile JavaBean & OTP Property

Problem Statement

CineHub's checkout screen auto-generates its form using a JavaBean-scanning framework. Build `MovieBookingProfile` as a fully compliant JavaBean, with a small convenience constructor for the common case, and an OTP property that's write-only for good.

Requirements

- Provide a public no-argument constructor, plus a convenience constructor that takes just a name and chains to it with `this(...)`.
- Every ordinary field needs a JavaBean-compliant `getX()/setX()` pair (`isConfirmed()`, not `getConfirmed()`).
- `otp` must be settable via `setOtp(String otp)`, but never retrievable again, in any form.

Function Signature(s)

```
public MovieBookingProfile()
public MovieBookingProfile(String name)
String getName(); void setName(String name)
boolean isConfirmed(); void setConfirmed(boolean confirmed)
void setOtp(String otp) // no matching getter exists anywhere
```

Examples

<code>new MovieBookingProfile("Rahul Dev").getName()</code>	<code>"Rahul Dev"</code>

<code>MovieBookingProfile p = new MovieBookingProfile("Rahul Dev"); p.setConfirmed(true); p.isConfirmed()</code>	<code>true</code>

<code>p.setOtp("4471");</code>	<code>(no observable output – no method anywhere on the class can retrieve this value again)</code>

HINT The name-only constructor should call `this()` and then set the name — not repeat whatever the no-arg constructor already does.

CONSTRAINTS `otp` is a 4–6 digit numeric string.

Key Concepts Tested: *JavaBean `getX()/setX()/isX` naming convention, a small constructor chain with `this(...)`, a true write-only property.*

PROBLEM 5 • Intermediate → Advanced

Immutable Booking Receipt & Nightly Settlement

Problem Statement

Every confirmed booking gets a BookingReceipt that must never change once issued — it's what a customer waves at a support desk if something goes wrong. Build it as genuinely immutable, then reconcile a night's worth of receipts, including a group-booking variant, without ever crashing on a null entry.

Requirements

- BookingReceipt's fields must be final, the class itself final, and seatNumbers must be defensively copied both in and out.
- "Changing" a receipt (e.g., fixing a mis-typed seat) means returning a brand-new object via a with-style method.
- The nightly settlement processor must use instanceof to handle a GroupBookingReceipt differently from a regular one, and must never throw on a null batch entry.

Function Signature(s)

```
public BookingReceipt(String bookingId, String[] seatNumbers)
String[] getSeatNumbers()
BookingReceipt withUpdatedSeat(int index, String newSeat)
public GroupBookingReceipt(String bookingId, String[] seatNumbers, int groupSize)
static String processNightlySettlement(BookingReceipt[] receipts)
```

Examples

BookingReceipt b = new BookingReceipt("CH-1001", new String[]{"A1","A2"}); String[] seats = b.getSeatNumbers(); seats[0] = "X"; b.getSeatNumbers()[0]	"A1"

BookingReceipt updated = b.withUpdatedSeat(1, "A3"); b.getSeatNumbers() updated.getSeatNumbers()	["A1", "A2"] ["A1", "A3"]

--	--

<pre>processNightlySettlement({ new GroupBookingReceipt("CH-2002", new String[]{"B1","B2"}, 2), null, new BookingReceipt("CH-3003", new String[]{"C1"}) })</pre>	<pre>"2 processed 1 null skipped 1 group 1 individual"</pre>
---	--

HINT Prove the copy works in both directions separately — a correct constructor copy says nothing about whether the getter copies too.

CONSTRAINTS seatNumbers length up to 10. Batch size up to 2,000.

Key Concepts Tested: *True immutability with defensive copying, the "with" pattern, instanceof-based dispatch, null-safety in aggregation.*